

# [M-04] `s_inAMM` Update Can Overflow Causing Position Exit DOS

---

## Summary

---

The `s_inAMM` variable tracking tokens deployed in Uniswap AMM uses the same dangerous casting pattern as M-01. When `longAmount > shortAmount + s_inAMM`, the intermediate calculation becomes negative, causing the `uint256` cast to produce an astronomically large value. This prevents users from exiting their positions.

## Vulnerability Detail

---

### The Vulnerable Pattern

```
// CollateralTracker.sol:1032
s_inAMM = uint256(int256(uint256(s_inAMM)) + (shortAmount - longAmou
//      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//      DANGEROUS CASTING CHAIN

// CollateralTracker.sol:1080
s_inAMM = uint256(int256(uint256(s_inAMM)) - (shortAmount - longAmou
```

### Step-by-Step Breakdown

```

// Given:
uint128 s_inAMM = 10 ether;    // Small amount in AMM
int128 shortAmount = 5 ether;  // New short
int128 longAmount = 100 ether; // Large long being closed

// Calculate: int256(uint256(s_inAMM)) + (shortAmount - longAmount)
int256 intermediate = int256(uint256(s_inAMM)) + (int256(shortAmount
// intermediate = 10 + (5 - 100)
// intermediate = 10 - 95
// intermediate = -85 ether (NEGATIVE!)

// Cast to uint256
uint256 asUInt = uint256(intermediate);
// asUInt = 11579208923731619542357098500868790785326998466564056403
// (2^256 - 85 ether)

// toUInt128() will FAIL
// Either REVERT or TRUNCATE to garbage

```

## Attack Scenario

### 1. Initial State:

- Pool has moderate liquidity
- User opens large long position when pool is well-capitalized
- `s_inAMM` increases to reflect long position

### 2. Liquidity Drain:

- Other users withdraw deposits
- Shorts close their positions
- `s_inAMM` decreases significantly

### 3. Trap Sprung:

- Original user tries to close their long
- `longAmount > shortAmount + s_inAMM`
- Calculation goes negative
- Transaction reverts

### 4. Result:

- User cannot exit their position

- Funds locked until pool regains liquidity
- No guarantee pool will recover

## Impact

---

### Severity Analysis

| FACTOR      | ASSESSMENT                                  |
|-------------|---------------------------------------------|
| User impact | Cannot exit position                        |
| Fund risk   | Temporary lock (could become permanent)     |
| Likelihood  | Medium - requires liquidity changes         |
| Attack cost | Low - natural market conditions can trigger |

### Economic Attack Vector

A sophisticated attacker could:

1. Wait for a large long position to be opened
2. Systematically withdraw liquidity from the pool
3. Trap the long holder's position
4. Potentially profit from the victim's inability to exit

### Cascading Failure Risk

If multiple long positions become trapped:

- Pool liquidity further decreases
- More positions become unexitable
- Potential for systemic failure

## Proof of Concept

```
function test_M04_InAMMOverflow() public {
    uint128 s_inAMM = 10 ether;
    int128 shortAmount = 5 ether;
    int128 longAmount = 100 ether; // Much larger

    // Calculate intermediate value
    int256 intermediate = int256(uint256(s_inAMM)) +
        (int256(shortAmount) - int256(longAmount));
    // = 10 + (5 - 100) = -85 ether

    console.logInt(intermediate); // -85000000000000000000

    // Cast to uint256
    uint256 asUint = uint256(intermediate);
    console.log("uint256(negative) =", asUint);
    // 1157920892373161954235709850086879078532699846656405640393725

    assertTrue(asUint > type(uint128).max, "Exceeds uint128");
    // toUint128() will FAIL
}
```

## Test Output

```
≡ M-04: s_inAMM Update Overflow ≡

Initial s_inAMM: 10 ETH
shortAmount: 5 ETH
longAmount: 100 ETH

Intermediate calculation:
  int256(s_inAMM) + (short - long)
  = 10 + (5 - 100) = -85 ETH
-85000000000000000000

!!! Intermediate value is NEGATIVE !!!
uint256(negative) =
  115792089237316195423570985008687907853269984665640564039372584007913:

This exceeds uint128.max!
toUint128() will:
  - REVERT (SafeCast)
  - or TRUNCATE (unchecked)
```

## Run the PoC

```
FOUNDRY_PROFILE=poc forge test --match-contract M01_M04_Arithmetic_V
```

## Tools Used

---

- Manual code review
- Foundry for PoC development

## Recommended Mitigation

---

### Option 1: Track Net Position (Recommended)

```
// Instead of tracking absolute s_inAMM, track net delta
int128 internal s_netPositionDelta;

function updateInAMM(int128 shortAmount, int128 longAmount) internal
    s_netPositionDelta += (shortAmount - longAmount);
    // Signed arithmetic handles negatives properly

    // Ensure s_inAMM stays non-negative
    require(s_netPositionDelta ≥ 0, "Invalid AMM state");
}
```

### Option 2: Pre-Validate Before Update

```

function updateInAMM(int128 shortAmount, int128 longAmount) internal
    int256 delta = int256(shortAmount) - int256(longAmount);
    int256 newValue = int256(uint256(s_inAMM)) + delta;

    // Explicit validation
    if (newValue < 0) {
        revert InsufficientAMMBalance();
    }
    if (newValue > type(uint128).max) {
        revert AMMBalanceOverflow();
    }

    s_inAMM = uint128(uint256(newValue));
}

```

### Option 3: Use Saturating Arithmetic

```

function updateInAMM(int128 shortAmount, int128 longAmount) internal
    int256 delta = int256(shortAmount) - int256(longAmount);
    int256 newValue = int256(uint256(s_inAMM)) + delta;

    // Saturate at 0 (floor) instead of underflowing
    if (newValue < 0) {
        s_inAMM = 0;
        // Log warning or take corrective action
        emit AMMBalanceSaturated();
    } else {
        s_inAMM = uint128(uint256(newValue));
    }
}

```

### Critical Consideration

This fix must be coordinated with M-01 as they share the same root cause (dangerous casting chain). A unified approach to all `int256 → uint256 → uint128` patterns should be implemented.